

## Practical 5

### COS214

**Platform Name:**

CampusGuard

**Group Members:**

Angela Ramaboea - u25445392

Matshidiso Dibakoane - u25227506

Maeryn Pillay - u25146484

## TASK 1

### Platform Description

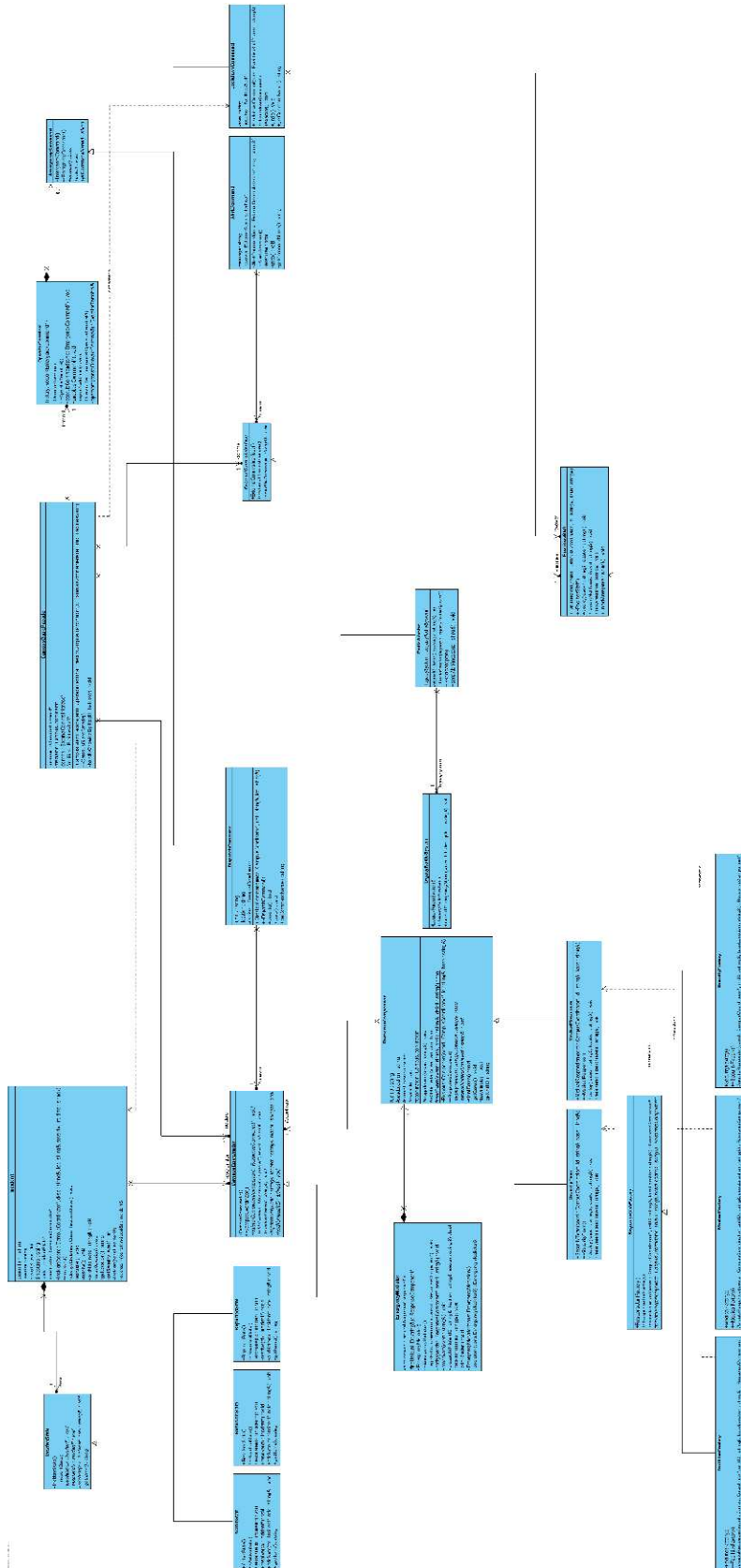
CampusGuard is an emergency-response coordination platform for a large university campus. Control-room operators use it to register emergency incidents such as fires or chemical spills, dispatch security, medical and facilities units, and carry out building-access actions such as locking and unlocking areas.

Each incident follows a defined lifecycle: Reported, Active and Resolved. What an incident allows depends on its current state, so only valid transitions can happen (for example, an incident cannot be resolved before it is active), and this is enforced without large conditional statements.

Response units never act in isolation or talk to each other directly. A central coordinator (mediator) relays events between them. When facilities staff lock or unlock an area, security teams secure or leave the perimeter. When an incident changes state, the units on scene deploy or stand down.

Operator actions (dispatching a unit, locking an area, issuing an alert) are encapsulated as command objects that can be undone and are recorded in an audit trail. A command that cannot be carried out, such as dispatching an unknown or already committed unit, is rejected and not recorded.

Off-campus alerts are sent through the city's legacy radio system, whose interface is incompatible with CampusGuard's, by way of an adapter. A facade provides a single entry point for the multi-step HAZMAT protocol for a chemical spill, while every subsystem remains directly usable.



**GoF Participant Identification**Command

- Command – EmergencyCommand
- Concrete Commands – DispatchCommand, LockdownCommand, AlertCommand
- Invoker – OperatorTerminal
- Receivers – CampusCoordinator (DispatchCommand), FacilitiesStaff (LockdownCommand), ExternalCommsInterface (AlertCommand)
- Client – main.cpp and CampusGuardFacade (both create commands and pass them to the terminal)

Mediator

- Mediator – CampusCoordinator
- Concrete Mediator – EmergencyMediator
- Colleague – ResponseComponent
- Concrete Colleagues – SecurityTeam, MedicalResponder, FacilitiesStaff
- Other users of the mediator (not colleagues) – Incident, DispatchCommand, CampusGuardFacade

Adapter

- Target – ExternalCommsInterface
- Adapter – RadioAdapter (object adapter)
- Adaptee – LegacyRadioSystem
- Clients – AlertCommand and CampusGuardFacade (both call sendAlert())

Facade

- Facade – CampusGuardFacade
- Subsystems – Incident, ExternalCommsInterface, CampusCoordinator, OperatorTerminal, FacilitiesStaff
- Client – main.cpp

## Two Additional Selected Patterns and Justification

State

- Context – Incident
- State – IncidentState
- Concrete States – ReportedState, ActiveState, ResolvedState
- Clients – main.cpp, CampusGuardFacade

*Justification:* Whether an incident may be escalated or resolved, and what escalating it means, depends on its current status. Each concrete state implements

only the behaviour valid for that status: ReportedState moves to Active, ActiveState raises severity or moves to Resolved, and ResolvedState rejects further changes. Invalid requests are therefore rejected with a clear message.

The simpler alternative, a status field plus switch statements in every Incident method, would duplicate the transition rules across methods and grow with every new status. With State, adding a status means adding one class. Every transition is also announced to the mediator, which is how response units react when an incident's condition changes.

### Factory Method

- Creator – ResponseUnitFactory (creator operation commissionUnit(), factory method createResponder())
- Concrete Creators – SecurityFactory, MedicalFactory, FacilitiesFactory
- Product – ResponseComponent
- Concrete Products – SecurityTeam, MedicalResponder, FacilitiesStaff
- Client – main.cpp

*Justification:* Bringing a new unit onto the response network always involves the same two steps: create the unit, then register it with the coordinator. Only the kind of unit varies. commissionUnit() fixes that sequence, so no unit can ever be left unregistered, and it defers only the variable step to createResponder() in each concrete factory.

The simpler alternative, calling new SecurityTeam(...) and registerComponent() directly in main, would repeat the registration step everywhere a unit is created and tie the client to every concrete unit class. With Factory Method, a new type of responder needs only a new product class and a new factory, and units can be commissioned mid-incident (MED-02 in scenario 2).

## **Associations, Inheritance, Multiplicities and Ownership**

### Associations and multiplicities

- EmergencyMediator ♦ ResponseComponent: composition, 1 to 0..\*. The mediator owns every registered unit.
- OperatorTerminal ♦ EmergencyCommand: composition, 1 to 0..\* (the command history).
- Incident ♦ IncidentState: composition, 1 to 1 (the current state).
- ResponseComponent → CampusCoordinator and Incident → CampusCoordinator: non-owning associations, many to 1.
- CampusGuardFacade → OperatorTerminal, CampusCoordinator, ExternalCommsInterface and FacilitiesStaff: non-owning associations, 1 to 1 each.
- Each concrete command → its receiver: non-owning association, to 1.

- RadioAdapter → LegacyRadioSystem: non-owning association, 1 to 1.
- «create» dependencies: each concrete factory creates its product, and CampusGuardFacade creates LockdownCommand. CampusGuardFacade also depends on Incident, which it receives as a parameter.

### Inheritance

- SecurityTeam, MedicalResponder and FacilitiesStaff inherit from the abstract ResponseComponent.
- EmergencyMediator implements CampusCoordinator.
- DispatchCommand, LockdownCommand and AlertCommand inherit from EmergencyCommand.
- ReportedState, ActiveState and ResolvedState inherit from IncidentState.
- RadioAdapter implements ExternalCommsInterface.
- SecurityFactory, MedicalFactory and FacilitiesFactory inherit from ResponseUnitFactory.

### Ownership and lifetimes

- main.cpp owns the mediator, operator terminal, facade, incidents, radio adapter and legacy radio system, and deletes them in a fixed order at shutdown.
- EmergencyMediator owns every response unit handed to it by commissionUnit() and deletes them in its destructor.
- OperatorTerminal takes ownership of every command passed to executeCommand(). A failed command is deleted immediately, an undone command is deleted after undo(), and the rest are deleted in the terminal's destructor.
- Incident owns its current IncidentState, deleting the old state on each transition and the current one in its destructor.
- All other pointers are non-owning: the facade's members, command receivers, the adapter's adaptee, and each unit's and incident's coordinator.
- Owning classes disable copying so ownership can never be duplicated.
- Every polymorphic base class has a virtual destructor, and Valgrind reports all heap blocks freed.

### **Design Rationale**

CampusGuard is built around one coordination hub. Response units (SecurityTeam, MedicalResponder, FacilitiesStaff) never hold pointers to one another. Every event passes through the CampusCoordinator (Mediator). With four or more units, direct links would create many-to-many coupling. Routing everything through one

coordinator means a new kind of unit only needs its own class and its own reactions in `receiveNotification()`. Each colleague decides how it reacts to an event, so the mediator relays events without any type checks.

The remaining patterns connect to this hub rather than working alongside it:

- **State:** Incident delegates `escalate`, `resolve` and `addNote` to its current state object, so invalid transitions are rejected without switch statements. Every real transition is broadcast through the coordinator, so units deploy or stand down when an incident's condition changes.
- **Command:** operator actions are objects, so the `OperatorTerminal` can execute, record, audit and undo them without knowing their receivers. A command that fails (an unknown or busy unit) returns false and is never recorded. `LockdownCommand`'s receiver, `FacilitiesStaff`, raises an event through the mediator when it locks an area, so a single operator action leads to coordination between units.
- **Adapter:** `RadioAdapter` confines the incompatible legacy radio API to one class. The rest of the system only ever calls `sendAlert(message)`, and only the adapter knows about signal codes and `transmitEmergencySignal()`.
- **Facade:** `handleChemicalSpill()` fixes the correct order of a four-step protocol that would otherwise have to be driven by the client. The subsystems stay public so the operator retains fine-grained control.
- **Factory Method:** `commissionUnit()` guarantees every new unit is registered with the coordinator. Only the choice of unit type is deferred to the concrete factories.

**Trade-off:** main keeps one typed `FacilitiesStaff*`, because that unit is also `LockdownCommand`'s receiver and the command needs its lock and unlock operations. This is the only place the client knows a concrete product class. We accepted it rather than adding a separate access-control interface for a single receiver.

## Pattern Interactions

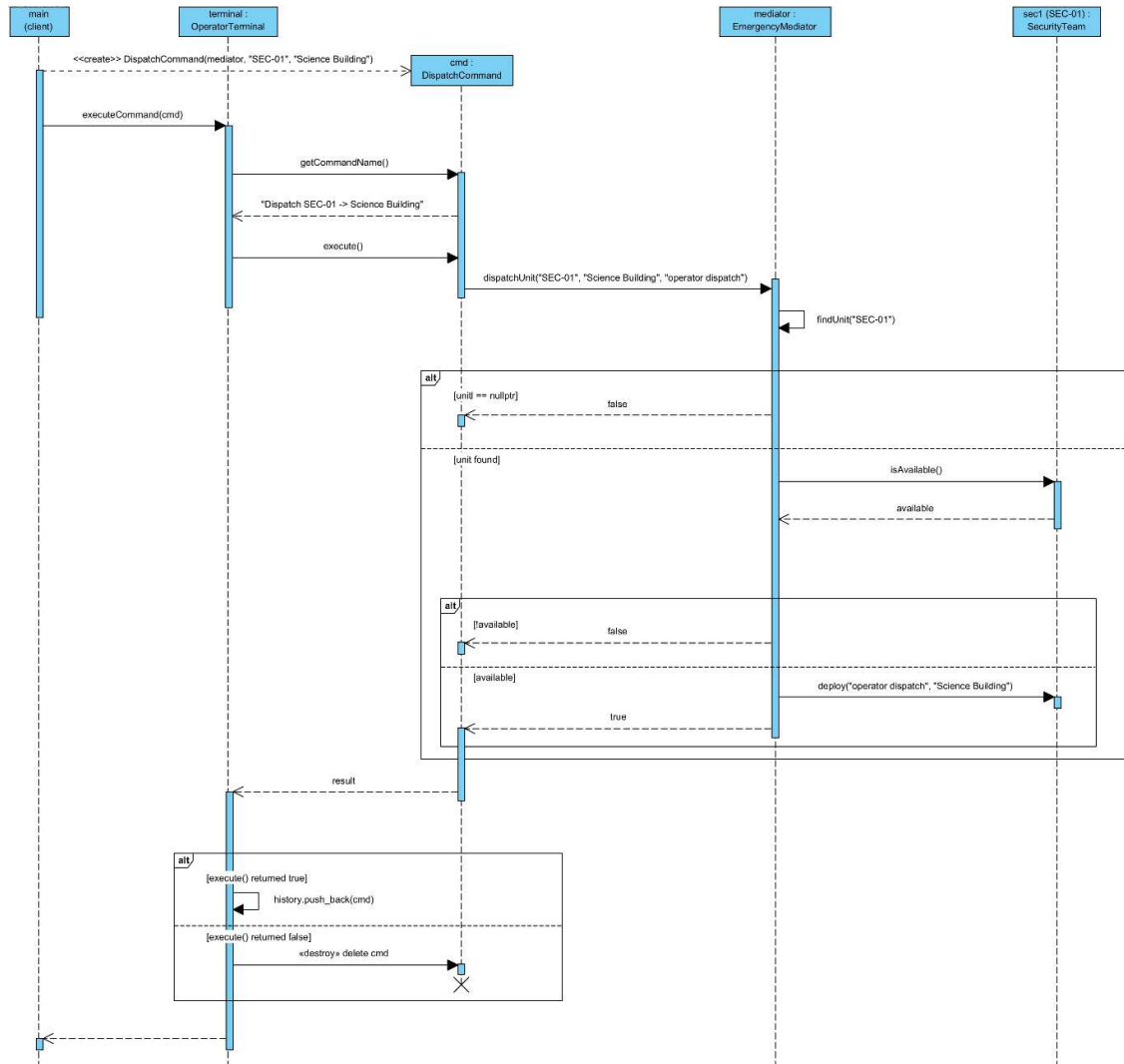
The six patterns form one execution flow rather than working independently.

- Factory Method → Mediator. Factory Method builds the response network: `commissionUnit()` creates each unit and registers it with the mediator.
- State → Mediator. When an incident changes state, the Incident broadcasts the change through the mediator. For example, when a fire becomes Active, facilities staff deploy to open emergency exits; when it is Resolved, the units on scene stand down.
- Command → Mediator. Commands pass through the operator terminal to their receivers. `DispatchCommand`'s receiver is the mediator itself, which checks that the unit exists and is available before deploying it.

LockdownCommand's receiver is FacilitiesStaff, whose lockArea() raises an event through the mediator, so a security team secures the perimeter. This is the sequence in which a command causes other components to coordinate.

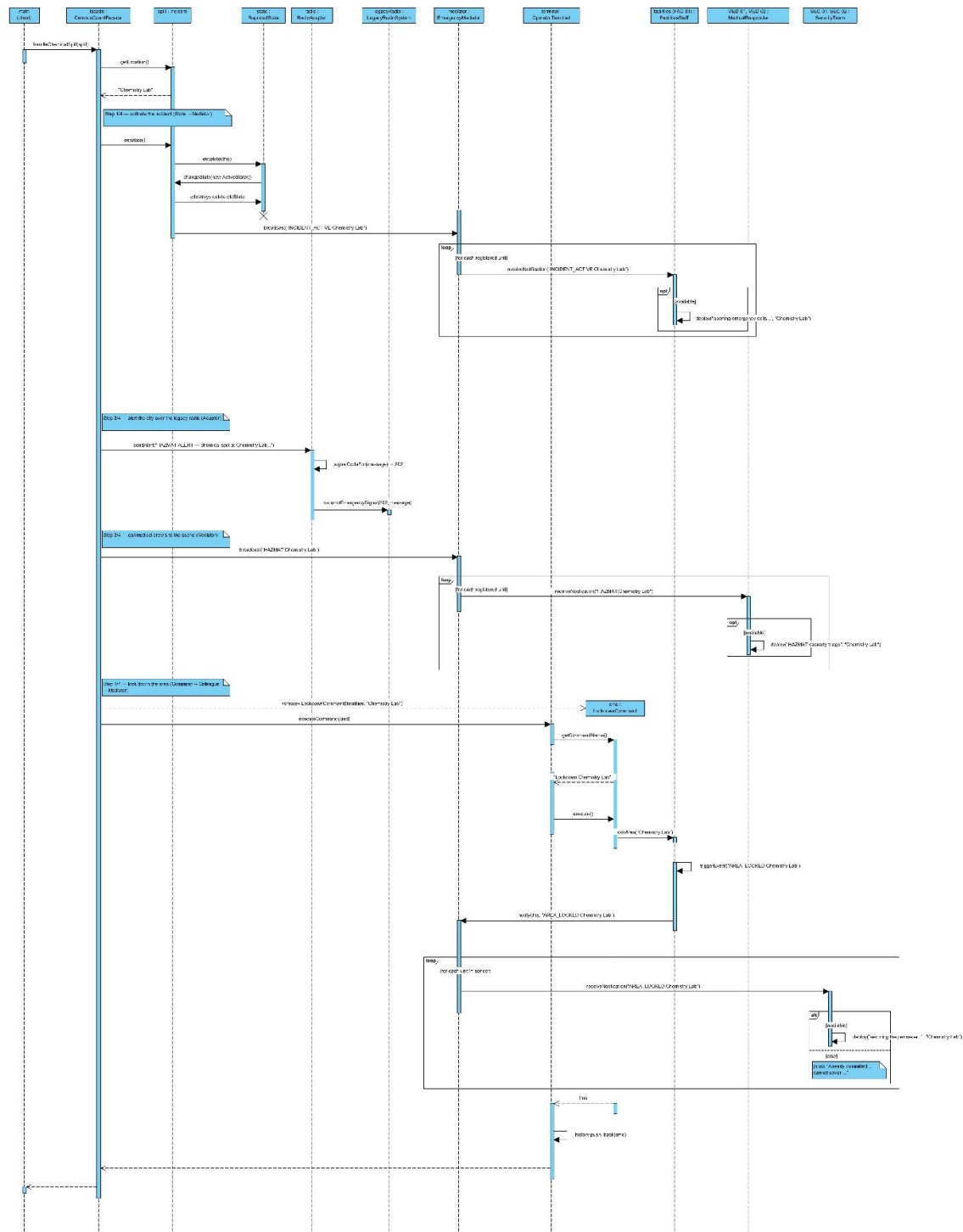
- Command → Adapter. AlertCommand's receiver is the ExternalCommsInterface, so the alert travels through the RadioAdapter, which works out the numeric signal code and calls the legacy transmitEmergencySignal().
- Facade → all of the above. CampusGuardFacade::handleChemicalSpill() ties the patterns together in one call. It activates the incident (State, which notifies the Mediator), alerts the city over the legacy radio (Adapter), calls medical crews to the scene (Mediator), and locks the area through the operator terminal (Command, which leads to FacilitiesStaff and then the Mediator).

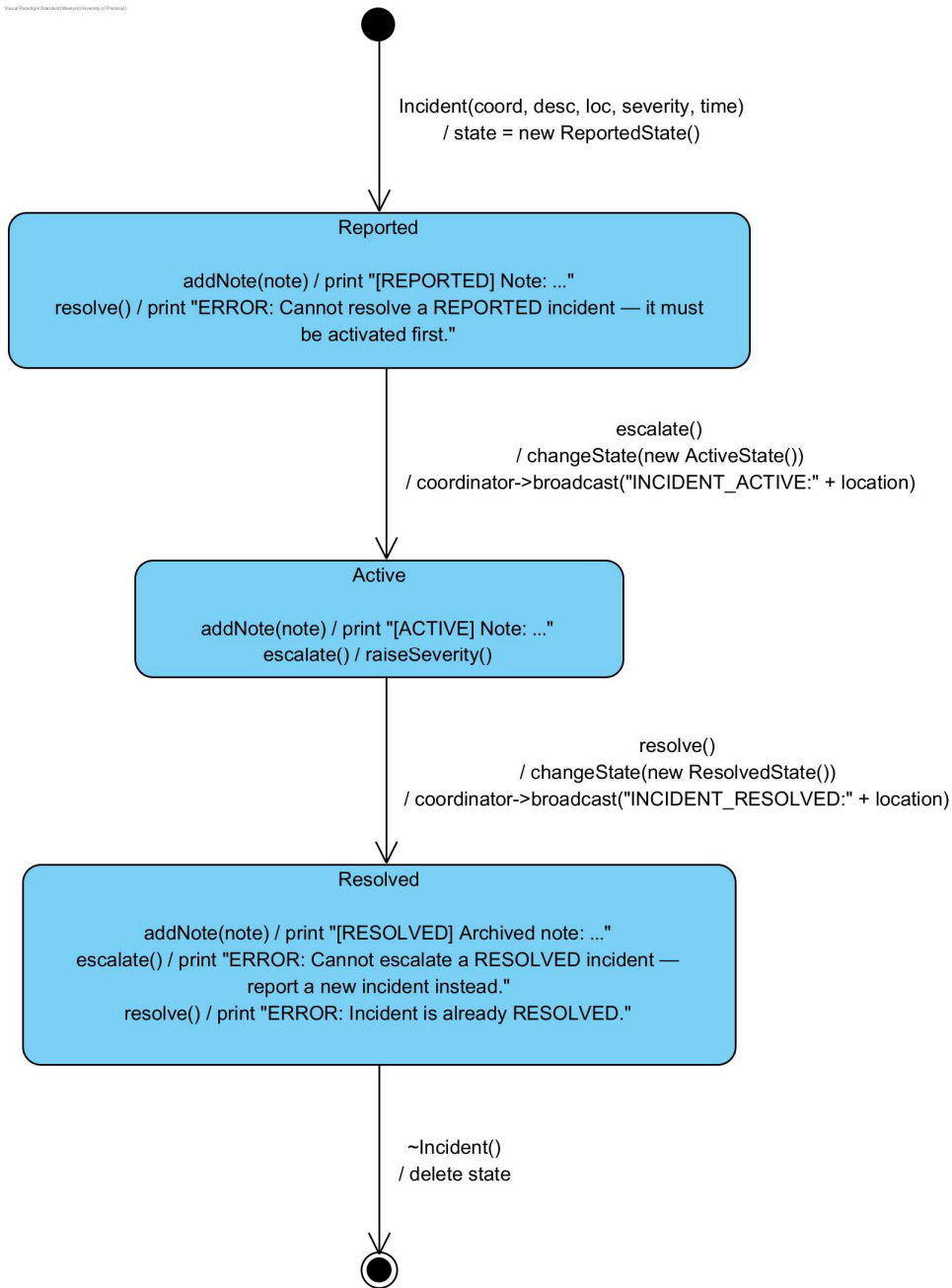
## TASK 4

**Sequence Diagram 1**Command + Mediator Sequence Diagram (Dispatch command)

### Sequence Diagram 2

## Facade + Adapter + State + Mediator + Command Sequence Diagram (HAZMAT protocol)



**State Diagram 1**State (Incident lifecycle)Visual Paradigm Standard (Version 4.1.0) (University of Pretoria)

## TASK 5

## Valgrind Evidence

```
u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_5/Repo/214-Prac-5-MMA$ make valgrind
valgrind --leak-check=full --show-leak-kinds=all ./CampusGuard
==907== Memcheck, a memory error detector
==907== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==907== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==907== Command: ./CampusGuard
==907==

=====
CAMPUSGUARD - SYSTEM START-UP
=====

--- [Mediator] Creating the coordination network ---
```

...

```
[OperatorTerminal] ===== AUDIT TRAIL =====
[OperatorTerminal] 1. Dispatch SEC-01 -> Science Building
[OperatorTerminal] 2. Alert "FIRE ALERT - Science Building. Evacuate. Do not use the lifts."
[OperatorTerminal] =====

=====
CAMPUSGUARD - SHUTDOWN
=====

All resources released. CampusGuard shut down cleanly.
==907==
==907== HEAP SUMMARY:
==907==   in use at exit: 0 bytes in 0 blocks
==907== total heap usage: 189 allocs, 189 frees, 82,342 bytes allocated
==907==
==907== All heap blocks were freed -- no leaks are possible
==907==
==907== For lists of detected and suppressed errors, rerun with: -s
==907== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

## GDB Debugging

```

u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_5/Repo/Prac_5$ gdb ./CampusGuard
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./CampusGuard...
(gdb) run
Starting program: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_5/Repo/Prac_5/CampusGuard

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

=====
CampusGuard - Emergency Response Coordination System
=====

[SYSTEM INITIALISATION]

[Setup] Creating EmergencyMediator...
[Setup] Spawning response units via factories...
[SecurityFactory] Instantiating SecurityTeam: SEC-01 @ Main Gate

Program received signal SIGSEGV, Segmentation fault.
ResponseComponent::ResponseComponent (this=0x55555557f750, coord=0x0, id="SEC-01", loc="Main Gate")
    at ResponseComponent.cpp:9
9      coordinator->registerComponent(this);
(gdb) backtrace
#0  ResponseComponent::ResponseComponent (this=0x55555557f750, coord=0x0, id="SEC-01", loc="Main Gate")
    at ResponseComponent.cpp:9
#1  0x00005555555d9000 in SecurityTeam::SecurityTeam (this=0x55555557f750, coord=0x0, id="SEC-01", loc="Main Gate")
    at SecurityTeam.cpp:7
#2  0x0000555555560527 in SecurityFactory::createResponder (this=0x7ffffffda28, coord=0x0, unitID="SEC-01",
startLocation="Main Gate") at SecurityFactory.cpp:10
#3  0x00005555555862b in main () at main.cpp:87
(gdb) frame 1
#1  0x00005555555d9000 in SecurityTeam::SecurityTeam (this=0x55555557f750, coord=0x0, id="SEC-01", loc="Main Gate")
    at SecurityTeam.cpp:7
7      : ResponseComponent(coord, id, loc) {}
(gdb) print receiver
$1 = (receiver_fct) 0x0
(gdb) |

```

During development, we encountered a critical execution bug where the system crashed with a seg fault during the initialisation phase. We used GDB to investigate the core dump.

By running backtrace we identified that the crash originated inside the ResponseComponent constructor at coordinator->registerComponent(this). We navigated up the call stack and observed that SecurityFactory::createResponder was passing a null pointer into the constructor. We realised that in main.cpp, we had initialised the EmergencyMediator as a null pointer before attempting to spawn our response units. When the unit attempted to register itself with the mediator, it dereferenced the null pointer, causing the crash. We resolved the bug by ensuring the EmergencyMediator is properly dynamically allocated before any factories are invoked.

**GitHub Repository**

[<https://github.com/MaerynPillay-30/214-Prac-5-MMA.git>]

**Collaboration Reflection**

We divided the work by pattern and by deliverable so each member owned a clear part of the system:

- Angela implemented the Mediator, Command, Adapter and Facade code and the scenarios.
- Maeryn led the design and implemented State and Factory Method.
- Matshidiso produced the behavioural diagrams and the GDB and Valgrind evidence.

Each member developed on their own branch and merged through pull requests. That kept the main branch buildable, and the Makefile and Docker setup let everyone run the same build.

Our main challenge was keeping the UML and the code in step. Once the patterns were integrated, the implementation had moved ahead of our first class diagram. Ownership lines were drawn the wrong way round, and some methods were declared but never used. We fixed this by tracing every diagram relationship and message back to the C++ source, then updating both. We removed unused code, corrected the compositions, and redrew the sequence and state diagrams from the final implementation.

Next time we would agree on the ownership policy and the interfaces between patterns before splitting up the coding, and update the diagrams with every merge rather than at the end.

## Contribution Statements

**Angela Ramaboea (u25445392):** Implemented the Mediator (CampusCoordinator, EmergencyMediator) and the response units (ResponseComponent, SecurityTeam, MedicalResponder, FacilitiesStaff). Implemented the Command pattern (EmergencyCommand, DispatchCommand, LockdownCommand, AlertCommand and the OperatorTerminal invoker). Implemented the Adapter (ExternalCommsInterface, RadioAdapter, LegacyRadioSystem) and the Facade (CampusGuardFacade). Wrote the two end-to-end scenarios in main.cpp.

**Matshidiso Dibakoane (u25227506):** Produced the three behavioural diagrams: sequence diagram 1 (dispatch command), sequence diagram 2 (HAZMAT facade workflow) and the State pattern state machine diagram. Conducted the GDB investigation and the Valgrind memory checks for the final executable. Set up the build and run environment (Makefile, Dockerfile, docker-compose.yml and README).

**Maeryn Pillay (u25146484):** Designed the system and produced the final UML class diagram. Wrote the Task 1 analysis: system description, GoF participant mapping, pattern justifications, and the associations, multiplicities and ownership decisions. Implemented the State pattern (Incident, IncidentState, ReportedState, ActiveState, ResolvedState) and the Factory Method pattern (ResponseUnitFactory, SecurityFactory, MedicalFactory, FacilitiesFactory).